

SIMATS ENGINEERING

ASSESSMENT TOOL 3

Shaik. Sameena(192372264)

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

Code of Conduct:

I Shaik. Sameena certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response

Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand
-----------------------------------	-----------	---	----------------------------------	---------------------------------------	-------------------------

Question 1 – Responsive Web Design

Responsive Web Design (RWD) is an approach to web development in which a website automatically adjusts its layout, size, images, and content according to the screen size and device being used. It allows the same website to work properly on mobile phones, tablets, laptops, and desktops.

Importance of Responsive Web Design

Responsive design is important because:

1. It provides a consistent user experience on different devices.
2. It improves readability and navigation on small screens.
3. It avoids creating separate websites for mobile and desktop.
4. It improves accessibility and user satisfaction.
5. It helps websites perform better in search engines.

Key CSS Techniques

The main CSS techniques used for responsiveness are:

- **Flexible widths:** Using %, vw, and max-width instead of fixed widths.
- **Flexible images:** Using max-width: 100% so images do not overflow.
- **Media queries:** Applying different styles for different screen sizes.
- **Flexbox:** Arranging elements in flexible rows or columns.
- **CSS Grid:** Creating responsive rows and columns.
- **Relative units:** Using rem, %, em, vw, and vh.

Example Using Flexbox

```
<div class="container">
  <div class="box">Home</div>
  <div class="box">About</div>
  <div class="box">Services</div>
</div>

.container {
  display: flex;
  gap: 20px;
```

```
    flex-wrap: wrap;
}
```

```
.box {
    flex: 1;
    min-width: 200px;
    padding: 30px;
    text-align: center;
    background: lightblue;
}
```

```
@media (max-width: 600px) {
    .container {
        flex-direction: column;
    }
}
```

On a desktop, the boxes can appear in a row. On a mobile device, the media query changes the layout into a column.

Improvements

The user experience can be improved by using readable font sizes, touch-friendly buttons, optimized images, simple navigation, sufficient spacing, and testing the website on different screen sizes.

Question 2 – Client-Side Validation

Client-side validation is the process of checking user input in the browser before the form is submitted to the server. It helps ensure that the information entered by the user is complete and follows the required format.

Purpose of Client-Side Validation

Client-side validation:

1. Prevents incorrect data from being submitted.
2. Provides immediate feedback to the user.
3. Reduces unnecessary server requests.
4. Improves the overall user experience.
5. Ensures that required fields are completed correctly.

JavaScript Event Handling

JavaScript event handling allows validation code to execute when an event occurs. Common events include submit, click, input, and blur.

For form validation, the submit event can be used. JavaScript checks the input values and prevents submission if the values are invalid.

Role of Regular Expressions

A **regular expression (RegEx)** is a pattern used to match and validate text. It can be used to check whether an email address or phone number follows a particular format.

Example

```
<form id="myForm">
  <input type="text" id="email" placeholder="Enter email">
  <input type="text" id="phone" placeholder="Enter phone number">
  <button type="submit">Submit</button>
</form>

<script>
```

```
document.getElementById("myForm").addEventListener("submit",
function(event) {

    let email = document.getElementById("email").value;
    let phone = document.getElementById("phone").value;

    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    let phonePattern = /^[0-9]{10}$/;

    if (!emailPattern.test(email)) {
        alert("Enter a valid email address");
        event.preventDefault();
    }

    if (!phonePattern.test(phone)) {
        alert("Enter a valid 10-digit phone number");
        event.preventDefault();
    }
});
</script>
```

The submit event runs the validation function. The test() method checks whether the input matches the regular expression. event.preventDefault() stops the form from being submitted when the input is invalid.

Improvements

Usability can be improved by displaying error messages near the corresponding fields instead of using only alerts. Input fields can also use suitable placeholders and HTML attributes such as required and type="email". For security, client-side validation should **not** be considered sufficient because it can be bypassed. The server must always perform its own validation and sanitization.

Question 3 – CSS Box Model and Positioning

The **CSS box model** describes how every HTML element is represented as a rectangular box. It consists of four main components:

1. **Content** – The actual text or other content inside the element.
2. **Padding** – Space between the content and border.
3. **Border** – A line surrounding the padding and content.
4. **Margin** – Space outside the border separating the element from other elements.

The total size of an element is affected by its content, padding, border, and margin.

CSS Positioning Techniques

CSS provides several positioning methods:

- **Static:** The default positioning method.
- **Relative:** Moves an element relative to its normal position.
- **Absolute:** Positions an element relative to its nearest positioned ancestor.
- **Fixed:** Positions an element relative to the browser window.
- **Sticky:** Allows an element to behave normally until a particular scrolling position is reached.

Incorrect use of absolute or fixed positioning can cause elements to overlap or move outside their containers when the browser window is resized.

Causes of Layout Breaking

A webpage may break during resizing because of:

- Fixed widths and heights.
- Large images that do not scale.
- Excessive use of absolute positioning.
- Fixed font sizes.
- Elements without proper wrapping.
- Incorrect margins and padding.

- Lack of media queries.
- Poorly designed Flexbox or Grid layouts.

Using Media Queries

Media queries allow CSS rules to change according to the screen size.

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

```
.box {  
  width: 300px;  
}
```

```
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

```
.box {  
  width: 100%;  
}  
}
```

For screens smaller than 768px, the boxes are arranged vertically and occupy the available width.

Best Practices

To maintain a stable and responsive layout:

1. Prefer flexible units such as %, rem, vw, and fr.
2. Use Flexbox or Grid for page layouts.
3. Avoid unnecessary fixed widths and heights.

4. Use max-width: 100% for images.
5. Use media queries for different screen sizes.
6. Use the box-sizing property.

```
* {  
  box-sizing: border-box;  
}
```

7. Test the webpage on mobile, tablet, and desktop screen sizes.

Question 4 – Document Object Model (DOM)

The **Document Object Model (DOM)** is a programming representation of an HTML document. The browser converts the HTML document into a tree-like structure in which elements such as headings, paragraphs, buttons, and forms become objects called DOM elements.

JavaScript can access and modify these elements using the DOM. This allows webpages to update their content dynamically without reloading the entire page.

Role of DOM

The DOM allows JavaScript to:

- Change HTML content.
- Change CSS styles.
- Add or remove elements.
- Modify attributes.
- Handle user events.
- Update webpage content dynamically.

Example

```
<h2 id="message">Welcome</h2>
```

```
<button id="button">Change Message</button>
```

```
<script>
```

```
document.getElementById("button").addEventListener("click", function() {  
    document.getElementById("message").textContent =  
        "Welcome to the webpage!";  
});
```

```
</script>
```

Initially, the webpage displays **Welcome**. When the user clicks the button, JavaScript finds the element with the ID message and changes its text using `textContent`.

Thus, the content is updated without refreshing the complete webpage.

Improving Performance and User Experience

DOM-based applications can be improved by:

1. Updating only the required elements instead of the entire page.
2. Avoiding unnecessary DOM manipulations.
3. Using event delegation for many similar elements.
4. Using efficient selectors.
5. Showing loading indicators when dynamic data takes time to load.
6. Providing immediate feedback to users.
7. Using asynchronous techniques such as `fetch()` for retrieving data without reloading the page.

Therefore, the DOM provides an important connection between HTML and JavaScript and makes modern interactive webpages possible.